C-PROGRAMMING (CSA0202)

CO3 ASSIGNMENT

Searching, Sorting, Arrays, Strings and Algorithm Design

NAME: LEELAVATHI R

REG.NO: 1925260191

Q1 – Searching a Library Catalogue

1. Linear Search and Binary Search

Linear search checks each element of an unsorted array from the first element until the key is found or the array ends. Binary search works only on a sorted array. It compares the key with the middle element and repeatedly discards the half that cannot contain the key.

2. Pseudocode

```
LINEAR_SEARCH(A, n, key):
  comparisons = 0
  for i = 0 to n-1:
    comparisons = comparisons + 1
    if A[i] == key: return i, comparisons
  return -1, comparisons
BINARY_SEARCH(A, n, key):  // A is sorted in ascending order
  low = 0; high = n-1; comparisons = 0
  while low <= high:
    mid = low + (high-low)/2
    comparisons = comparisons + 1
    if A[mid] == key: return mid, comparisons
    else if A[mid] < key: low = mid+1
    else: high = mid-1
  return -1, comparisons
```

3. Comparison Analysis

For n elements, linear search has best case 1 comparison, worst case n comparisons, and average (n+1)/2 comparisons for a successful search when each position is equally likely. An unsuccessful search requires n comparisons.

Binary search has best case 1 comparison. Its worst-case number of comparisons is at most floor(log2 n)+1. For n = 100,000, this is at most 17 comparisons. Linear search may require 100,000 comparisons. Binary search is therefore much faster for large sorted catalogues.

4. Why Sorting and the Midpoint Formula Matter

Binary search needs sorted data because it uses the order of the elements to decide which half can be discarded. Without sorting, that decision is not valid. The expression mid = low + (high-low)/2 is preferred to (low+high)/2 because low+high may overflow the range of an integer for very large indices.

5. Completed C Function

```
#include <stdio.h>


int steps = 0;


int binary_search(const int a[], int n, int key)
{
    int low = 0, high = n - 1;
```

```
    steps = 0;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        steps++;

        if (a[mid] == key)
            return mid;

        if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int a[100], n, key, result;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n <= 0 || n > 100)
    {
        printf("Invalid array size!\n");
        return 1;
    }

    printf("Enter %d elements in sorted order:\n", n);
```

```
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);

        if (i > 0 && a[i] < a[i - 1])
        {
            printf("Array must be sorted!\n");
            return 1;
        }
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    result = binary_search(a, n, key);

    if (result != -1)
        printf("Element found at position %d\n", result + 1);
    else
        printf("Element not found!\n");

    printf("Number of steps: %d\n", steps);

    return 0;
}
```

main.c

Clear Run ▶

```c

#include <stdio.h>

int steps = 0;

int binary_search(const int a[], int n, int key)
{
    int low = 0, high = n - 1;

    steps = 0;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        steps++;

        if (a[mid] == key)
            return mid;

        if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

int main()
{
    int a[100], n, key, result;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n <= 0 || n > 100)
    {
        printf("Invalid array size!\n");
        return 1;
    }

    printf("Enter %d elements in sorted order:\n", n);

    for (int i = 0; i < n; i++)
    {
```

Output

```
Enter number of elements: 4
Enter 4 elements in sorted order:
10 20 30 40
Enter element to search: 40
Element found at position 4
Number of steps: 3
```

6. Missing Book and Alternatives

A missing key may require the search to continue until the search interval becomes empty. This can take the maximum number of binary-search comparisons, just like a worst-case successful search. A hash table with linear probing gives O(1) average search time under a good hash distribution, but may take O(n) in the worst case and needs table space. Interpolation search has O(log log n) average time for uniformly distributed sorted keys, but O(n) worst-case time. Binary search consistently gives O(log n) time and uses O(1) extra space.

Q2 – Sorting the Examination Hall Seating Register

1. Sorting Algorithms

Bubble sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. An early-exit flag records whether a swap occurred during a pass. If no swap occurs, the array is already sorted and the algorithm stops.

Selection sort repeatedly finds the smallest remaining element and places it at the next position. It performs $O(n^2)$ comparisons in best, average and worst cases, with at most O(n) swaps.

Insertion sort builds a sorted portion one element at a time by shifting larger elements to the right and inserting the current element in its correct position. It is efficient for nearly sorted data because only a few shifts are needed.

2. Pseudocode

```
BUBBLE_SORT(A, n):
  for i = 0 to n-2:
    swapped = false
    for j = 0 to n-2-i:
      if A[j] > A[j+1]: swap(A[j], A[j+1]); swapped = true
    if swapped == false: break
SELECTION_SORT(A, n):
  for i = 0 to n-2:
    min = i
    for j = i+1 to n-1:
      if A[j] < A[min]: min = j
    if min != i: swap(A[i], A[min])
INSERTION_SORT(A, n):
  for i = 1 to n-1:
    key = A[i]; j = i-1
    while j >= 0 and A[j] > key:
      A[j+1] = A[j]; j = j-1
    A[j+1] = key
```

3. Complexity and Stability

Bubble sort with early exit: best O(n), average $O(n^2)$, worst $O(n^2)$; stable when equal elements are not swapped. Selection sort: $O(n^2)$ in all cases; usually not stable. Insertion sort: best O(n), average $O(n^2)$, worst $O(n^2)$; stable when only strictly larger elements are shifted.

Stability matters when students have equal marks because a stable sort preserves their previous relative order, such as registration order or another secondary arrangement.

4. Recommendation

For a register that is already sorted except for 20 late registrations, insertion sort is a suitable choice. It can insert each late registration into its correct position with relatively few shifts, making it efficient for nearly sorted data.

5. Counting Sort Alternative

If register numbers fall within a small known range k, counting sort counts the occurrences of each value and writes them back in order. Its time complexity is O(n+k) and extra space is O(k). It is unsuitable when k is extremely large compared with n or when the values are not integers in a manageable range.

```
#include <stdio.h>
void printArray(int a[], int n) {
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
void bubbleSort(int a[], int n) {
    int i, j, temp, swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

void selectionSort(int a[], int n) {
    int i, j, min, temp;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        temp = a[i]; a[i] = a[min]; a[min] = temp;
    }
}

void insertionSort(int a[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = a[i]; j = i - 1;
        while (j >= 0 && a[j] > key) {
```

```
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

int main(void) {
    int a[] = {45, 12, 78, 23, 56, 10};
    int b[] = {45, 12, 78, 23, 56, 10};
    int c[] = {45, 12, 78, 23, 56, 10};
    int n = 6;

    bubbleSort(a, n);
    selectionSort(b, n);
    insertionSort(c, n);

    printf("Bubble Sort:    "); printArray(a, n);
    printf("Selection Sort: "); printArray(b, n);
    printf("Insertion Sort: "); printArray(c, n);
    return 0;
}
```

main.c

Clear Run ▶

```c
#include <stdio.h>
void printArray(int a[], int n) {
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
void bubbleSort(int a[], int n) {
    int i, j, temp, swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

void selectionSort(int a[], int n) {
    int i, j, min, temp;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        temp = a[i]; a[i] = a[min]; a[min] = temp;
    }
}

void insertionSort(int a[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = a[i]; j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

int main(void) {
    int a[] = {45, 12, 78, 23, 56, 10};
    int b[] = {45, 12, 78, 23, 56, 10};
    int c[] = {45, 12, 78, 23, 56, 10};
    int n = 6;
```

Output

```
Bubble Sort:    10 12 23 45 56 78
Selection Sort: 10 12 23 45 56 78
Insertion Sort: 10 12 23 45 56 78
```

Q3 – Merging Two Merit Lists

1. Merge Algorithm

Two arrays are already sorted in descending order. Two indices point to the current elements. The larger score is copied to the output array, and the corresponding index is advanced. When one input ends, all remaining elements of the other input are copied.

```
MERGE(A, m, B, n, OUT):
  i = 0; j = 0; k = 0
  while i < m and j < n:
    if A[i] >= B[j]: OUT[k] = A[i]; i = i+1
    else: OUT[k] = B[j]; j = j+1
    k = k+1
  while i < m: OUT[k] = A[i]; i=i+1; k=k+1
  while j < n: OUT[k] = B[j]; j=j+1; k=k+1
  return k
```

2. Completed C Function

```
int merge(const int a[], int na, const int b[], int nb, int out[]) {
  int i=0, j=0, k=0;
  while (i<na && j<nb) {
    if (a[i] >= b[j]) out[k++] = a[i++];
    else out[k++] = b[j++];
  }
  while (i<na) out[k++] = a[i++];
  while (j<nb) out[k++] = b[j++];
  return k;
}
```

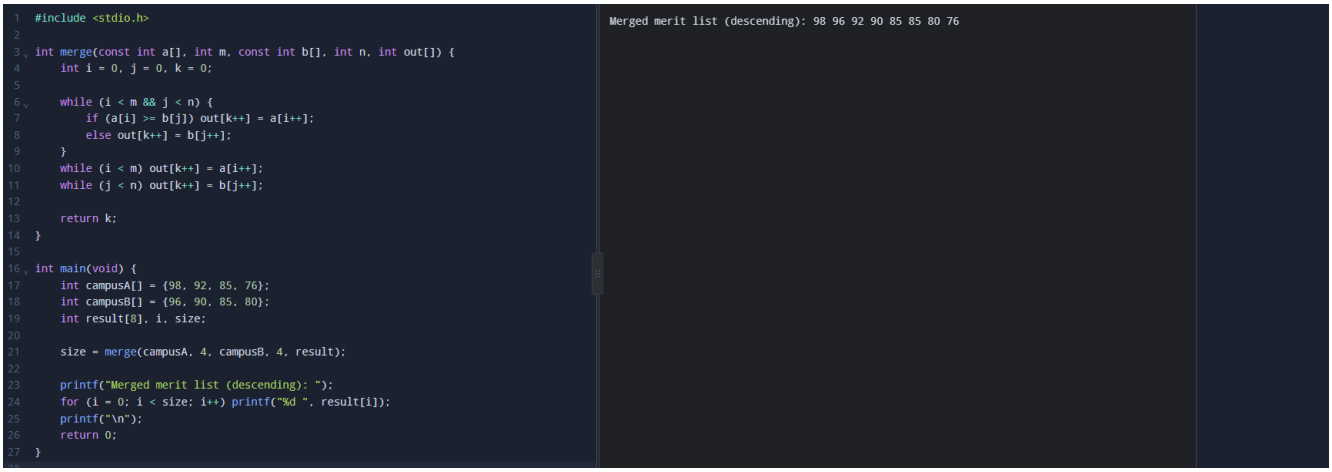
3. Complexity and Correctness

Merging m and n elements takes O(m+n) time and O(m+n) extra space for the output array. At each comparison, at least one element is copied and its index advances, so no element is repeatedly compared as the current element. The number of comparisons is at most m+n-1 when both arrays are non-empty. If one list is empty, the main loop is skipped and the other list is copied directly.

Using >= chooses the first campus when equal scores are encountered, preserving the order of equal scores from that input relative to the other input. If campus priority is required, this tie rule should be stated explicitly.

4. Comparing Alternatives

Copying both arrays together and sorting with an efficient comparison sort such as qsort generally takes O((m+n) log(m+n)) time. Merging takes O(m+n), so it is preferable when both lists are already sorted. A simple in-place merge can avoid a third array but may require shifting elements and can take O(m·n) time. It saves auxiliary array space at the cost of more processing.



```
#include <stdio.h>

int merge(const int a[], int m, const int b[], int n, int out[]) {
    int i = 0, j = 0, k = 0;

    while (i < m && j < n) {
        if (a[i] >= b[j]) out[k++] = a[i++];
        else out[k++] = b[j++];
    }
    while (i < m) out[k++] = a[i++];
    while (j < n) out[k++] = b[j++];

    return k;
}

int main(void) {
    int campusA[] = {98, 92, 85, 76};
    int campusB[] = {96, 90, 85, 80};
    int result[8], i, size;

    size = merge(campusA, 4, campusB, 4, result);

    printf("Merged merit list (descending): ");
    for (i = 0; i < size; i++) printf("%d ", result[i]);
    printf("\n");
    return 0;
}
```

```
Merged merit list (descending): 98 96 92 90 85 85 80 76
```

```
#include <stdio.h>

int merge(const int a[], int m, const int b[], int n, int out[]) {
    int i = 0, j = 0, k = 0;

    while (i < m && j < n) {
        if (a[i] >= b[j]) out[k++] = a[i++];
        else out[k++] = b[j++];
    }
    while (i < m) out[k++] = a[i++];
    while (j < n) out[k++] = b[j++];

    return k;
}

int main(void) {
    int campusA[] = {98, 92, 85, 76};
    int campusB[] = {96, 90, 85, 80};
    int result[8], i, size;

    size = merge(campusA, 4, campusB, 4, result);

    printf("Merged merit list (descending): ");
    for (i = 0; i < size; i++) printf("%d ", result[i]);
    printf("\\n");
    return 0;
}
```

Q4 – Two-Dimensional Arrays: District Rainfall Records

1. Array Representation

rain[38][12] stores 38 rows, one for each district, and 12 columns, one for each month. In C, two-dimensional arrays are stored in row-major order: all 12 monthly values of district 0 are stored first, followed by district 1, and so on.

If the base address is B and each element occupies w bytes, the address of rain[i][j] is B + ((i × 12) + j) × w.

2. Computations

```
For each district i:
  total[i] = 0
  for each month j from 0 to 11:
    total[i] += rain[i][j]

For each month j:
  column_total = 0
  for each district i from 0 to 37:
    column_total += rain[i][j]
  monthly_average[j] = column_total / 38.0

Track maximum rainfall while reading all elements.
```

To find deficient districts, first calculate the state average across all 38 × 12 values. A district is below 60% of the state average when its annual average (annual total divided by 12) is less than 0.60 multiplied by the state-wide average per district-month. The interpretation of 'district rainfall' should be kept consistent with the instructor's intended measure.

3. Row-Major Order and Performance

Row-wise traversal accesses adjacent memory locations and usually uses the CPU cache efficiently. Column-wise traversal jumps between rows, which may cause more cache misses for large arrays. Both methods visit the same number of elements, but row-wise traversal is generally faster in C.

A three-dimensional extension can be declared as rain[years][38][12], where each year contains 38 districts and each district contains 12 monthly values.

4. Why a 2-D Array?

A two-dimensional array groups related rainfall values in one structure, makes loops and calculations simpler, and provides contiguous storage. Thirty-eight separate arrays would be harder to manage and would make passing the complete dataset to functions less convenient.

A one-dimensional array using index i×12+j also provides O(1) access and can have the same contiguous layout. An array of 38 separately allocated row pointers also provides O(1) indexing but requires extra pointer storage and may have less predictable memory locality.

```
#include <stdio.h>


#define DISTRICTS 4
#define MONTHS 12
```

```
int main(void)
{
    double rain[DISTRICTS][MONTHS];
    double annual[DISTRICTS] = {0};
    double monthly[MONTHS] = {0};

    double stateTotal = 0, maximum = -1;

    int maxDistrict = 0, maxMonth = 0;
    int i, j;

    for (i = 0; i < DISTRICTS; i++)
    {
        printf("\nEnter 12 monthly rainfall values for District %d:\n", i + 1);

        for (j = 0; j < MONTHS; j++)
        {
            scanf("%lf", &rain[i][j]);

            annual[i] += rain[i][j];
            monthly[j] += rain[i][j];
            stateTotal += rain[i][j];

            if (rain[i][j] > maximum)
            {
                maximum = rain[i][j];
                maxDistrict = i + 1;
                maxMonth = j + 1;
            }
        }
    }

    printf("\nAnnual Rainfall Totals:\n");

    for (i = 0; i < DISTRICTS; i++)
```

```
        printf("District %d: %.2f\n", i + 1, annual[i]);

    printf("\nMonthly Average Rainfall:\n");

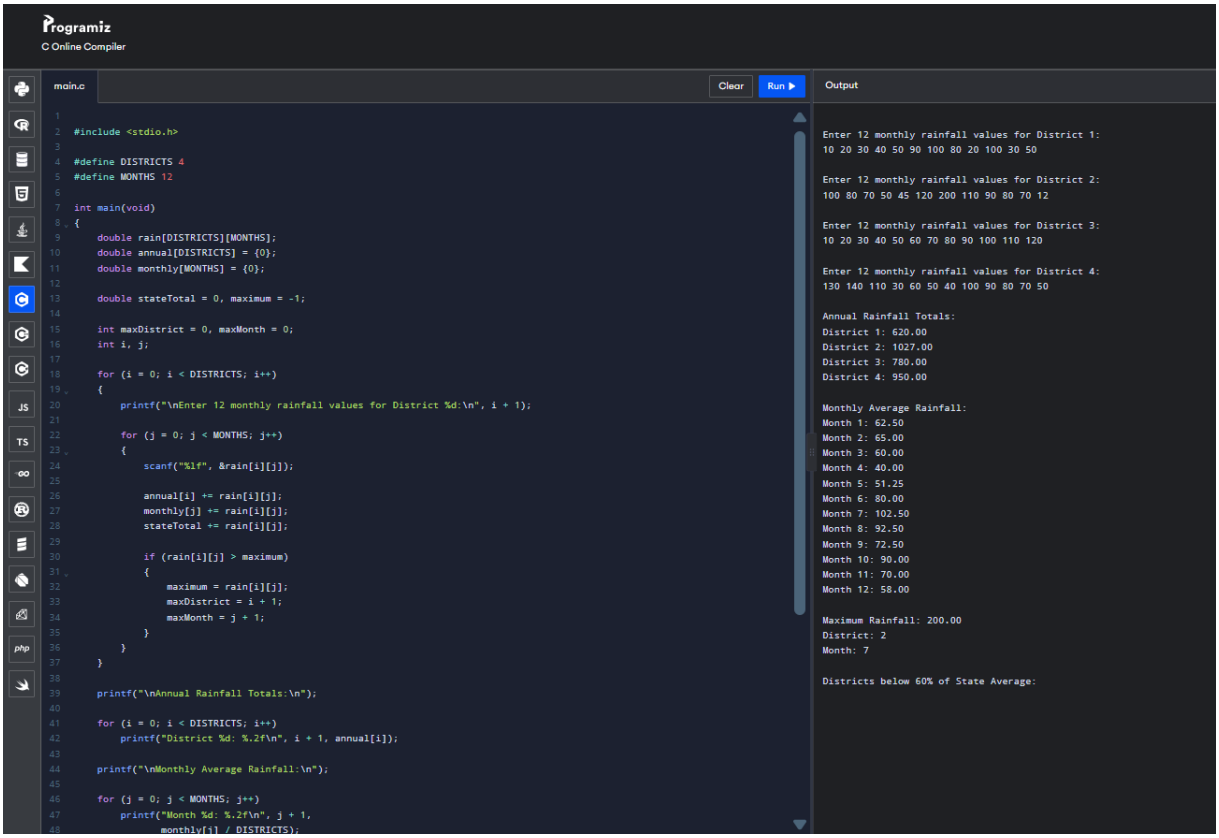
    for (j = 0; j < MONTHS; j++)
        printf("Month %d: %.2f\n", j + 1,
               monthly[j] / DISTRICTS);

    printf("\nMaximum Rainfall: %.2f", maximum);
    printf("\nDistrict: %d", maxDistrict);
    printf("\nMonth: %d\n", maxMonth);

    printf("\nDistricts below 60%% of State Average:\n");

    for (i = 0; i < DISTRICTS; i++)
    {
        if (annual[i] / MONTHS <
            0.60 * (stateTotal / (DISTRICTS * MONTHS)))
        {
            printf("District %d\n", i + 1);
        }
    }

    return 0;
}
```


Programiz
C Online Compiler
main.c
Clear
Run
Output
#include <stdio.h>
#define DISTRICTS 4
#define MONTHS 12
int main(void)
{
double rain[DISTRICTS][MONTHS];
double annual[DISTRICTS] = {0};
double monthly[MONTHS] = {0};
double stateTotal = 0, maximum = -1;
int maxDistrict = 0, maxMonth = 0;
int i, j;
for (i = 0; i < DISTRICTS; i++)
{
printf("\nEnter 12 monthly rainfall values for District %d:\n", i + 1);
for (j = 0; j < MONTHS; j++)
{
scanf("%lf", &rain[i][j]);
annual[i] += rain[i][j];
monthly[j] += rain[i][j];
stateTotal += rain[i][j];
if (rain[i][j] > maximum)
{
maximum = rain[i][j];
maxDistrict = i + 1;
maxMonth = j + 1;
}
}
}
printf("\nAnnual Rainfall Totals:\n");
for (i = 0; i < DISTRICTS; i++)
printf("District %d: %.2f\n", i + 1, annual[i]);
printf("\nMonthly Average Rainfall:\n");
for (j = 0; j < MONTHS; j++)
printf("Month %d: %.2f\n", j + 1,
monthly[j] / DISTRICTS);
Enter 12 monthly rainfall values for District 1:
10 20 30 40 50 90 100 80 20 100 30 50
Enter 12 monthly rainfall values for District 2:
100 80 70 50 45 120 200 110 90 80 70 12
Enter 12 monthly rainfall values for District 3:
10 20 30 40 50 60 70 80 90 100 110 120
Enter 12 monthly rainfall values for District 4:
130 140 110 30 60 50 40 100 90 80 70 50
Annual Rainfall Totals:
District 1: 620.00
District 2: 1027.00
District 3: 780.00
District 4: 950.00
Monthly Average Rainfall:
Month 1: 62.50
Month 2: 65.00
Month 3: 60.00
Month 4: 40.00
Month 5: 51.25
Month 6: 80.00
Month 7: 102.50
Month 8: 92.50
Month 9: 72.50
Month 10: 90.00
Month 11: 70.00
Month 12: 58.00
Maximum Rainfall: 200.00
District: 2
Month: 7
Districts below 60% of State Average:

Q5 – Processing Text Without the Library

1. Null Character and Word Count

A C string ends with the null character '\0'. It marks the end of the text and allows functions to know where the string stops. Word counting can be done by counting a character that is not a space when it follows the beginning of the string or a space.

```
WORD_COUNT(s):
  count = 0; inside = false
  for each character until '\0':
    if character is not a space:
      if inside == false: count++; inside = true
    else: inside = false
  return count
```

2. Completed C Functions

```
int word_count(const char s[]) {
  int count=0, inside=0;
  for (int i=0; s[i]!='\\0'; i++) {
    if (s[i] != ' ' && inside == 0) { count++; inside=1; }
    else if (s[i] == ' ') inside=0;
  }
  return count;
}
int is_palindrome(const char s[]) {
  int left=0, right=0;
  while (s[right] != '\\0') right++;
  right--;
  while (left < right) {
    while (left < right && !isalpha((unsigned char)s[left])) left++;
    while (left < right && !isalpha((unsigned char)s[right])) right--;
    if (tolower((unsigned char)s[left]) != tolower((unsigned char)s[right])) return 0;
    left++; right--;
  }
  return 1;
}
```

The palindrome function uses ctype.h for isalpha and tolower. It ignores punctuation and letter case. For the sample text, 'Madam, I'm Adam' is a palindrome, while 'Hello' is not.

3. Strings and Safety

char s[] = "OK" creates a modifiable character array containing O, K and '\0'. char *s = "OK" points to a string literal, which must not be modified. Recomputing strlen(s) in every loop condition can make a loop $O(n^2)$, because strlen scans the string each time. Store the length once to keep the traversal O(n).

Copying a long string into a small destination array using strcpy can overflow the destination buffer. Use a correctly sized destination and bounded input such as fgets. snprintf can limit the number of characters written. gets() was removed from the C standard because it cannot limit input length and can cause buffer overflows.

The word_count function above treats ordinary spaces as separators. A tab is not treated as a separator by that exact implementation; to support tabs and other whitespace, use isspace((unsigned char)s[i]) instead of checking only for ' '.

4. When to Use String Library Functions

Writing a custom string function is useful for learning, special rules or when library use is restricted. Standard library functions are preferable for normal applications because they are tested and reduce duplicated code. Safe buffer sizes and input handling are still essential.

```
#include <stdio.h>
#include <ctype.h>

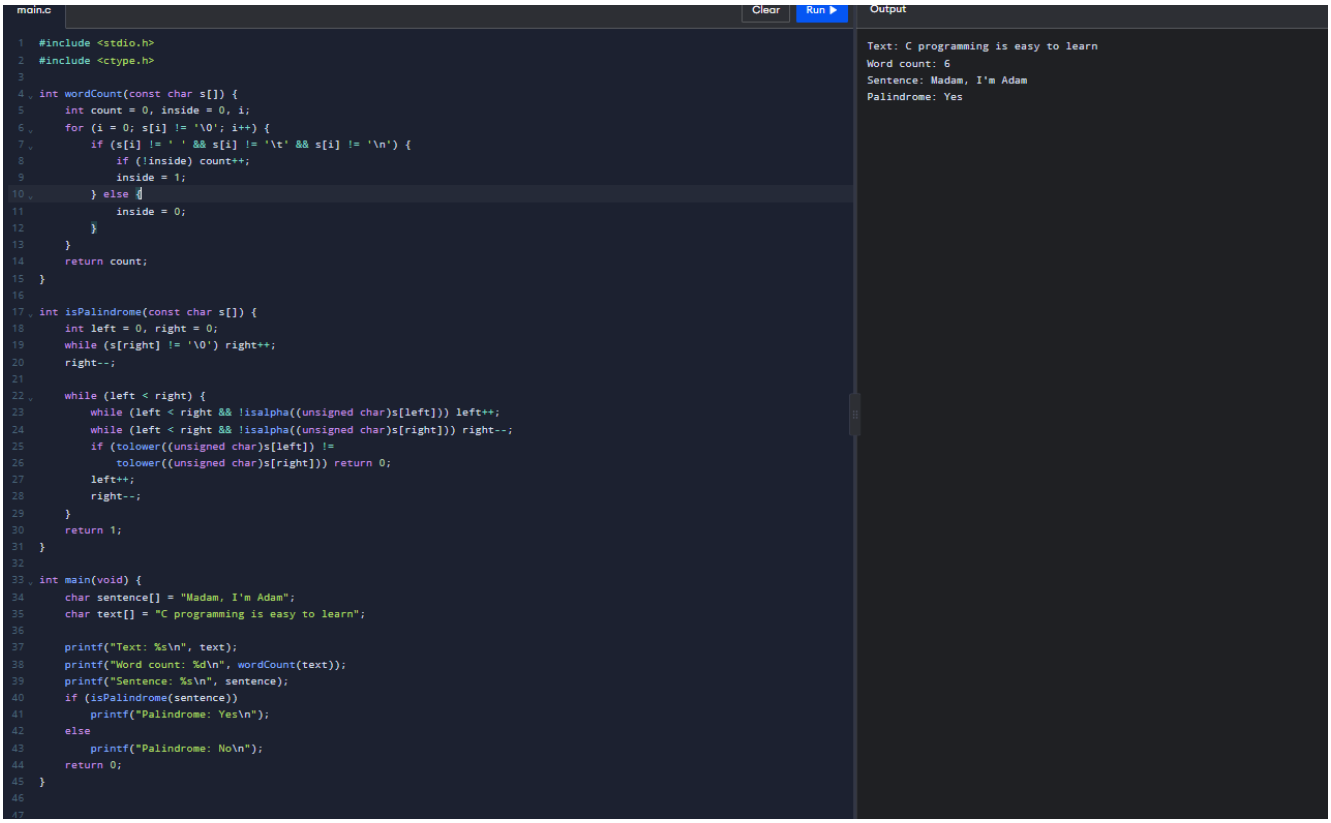
int wordCount(const char s[]) {
    int count = 0, inside = 0, i;
    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] != ' ' && s[i] != '\t' && s[i] != '\n') {
            if (!inside) count++;
            inside = 1;
        } else {
            inside = 0;
        }
    }
    return count;
}

int isPalindrome(const char s[]) {
    int left = 0, right = 0;
    while (s[right] != '\0') right++;
    right--;

    while (left < right) {
        while (left < right && !isalpha((unsigned char)s[left])) left++;
        while (left < right && !isalpha((unsigned char)s[right])) right--;
        if (tolower((unsigned char)s[left]) !=
            tolower((unsigned char)s[right])) return 0;
        left++;
        right--;
    }
    return 1;
}
```

```
int main(void) {
    char sentence[] = "Madam, I'm Adam";
    char text[] = "C programming is easy to learn";

    printf("Text: %s\n", text);
    printf("Word count: %d\n", wordCount(text));
    printf("Sentence: %s\n", sentence);
    if (isPalindrome(sentence))
        printf("Palindrome: Yes\n");
    else
        printf("Palindrome: No\\n");
    return 0;
}
```



Q6 – A Directory of Names: Sorting and Searching Strings

1. String Comparison and Sorting

strcmp compares two strings character by character using their character values. It returns a negative value if the first string comes before the second, zero if they are equal, and a positive value if the first comes after the second. Therefore, uppercase and lowercase forms such as 'Apple' and 'apple' may have different order because their character codes differ.

After sorting alphabetically, duplicate names appear next to one another. A scan of adjacent entries can identify duplicates. Binary search can find a name in O(log n) string comparisons after sorting.

2. Complexity

A comparison sort of n names uses O(n log n) string comparisons on average. Each comparison may inspect up to L characters, where L is the maximum name length, so the character-comparison work can be O(n log n × L). Swapping full names copies up to O(L) characters per swap. Sorting indices or pointers reduces the amount of data moved because only small integer indices or addresses are swapped.

3. Index/Pointer Sorting

Sorting an array of indices keeps the original names in place and rearranges only the indices. This can save time when names are long and avoids repeatedly copying character arrays. The trade-off is extra memory for the index or pointer array and an extra lookup through the index when accessing a name.

4. Input Length

With names[500][30], each row has room for at most 29 visible characters plus the terminating '\0'. A 29-character name can fit. A 30-character name cannot fit completely with its terminator and must be detected or safely truncated. fgets should be used, and its trailing newline should be removed when present.

5. Case-Insensitive Comparison and qsort

A case-insensitive comparison can compare tolower((unsigned char)s1[i]) and tolower((unsigned char)s2[i]) until a difference or the string end is found. qsort can sort an array of pointers using a comparator that dereferences the pointers and compares the names. The pointer array requires O(n) extra space; qsort typically performs O(n log n) comparisons, with each comparison potentially examining up to L characters.

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>


#define N 6

#define LEN 30


int compareIgnoreCase(const char a[], const char b[]) {

   int i = 0;

   while (a[i] && b[i]) {

      int x = tolower((unsigned char)a[i]);

      int y = tolower((unsigned char)b[i]);
```

```
        if (x != y) return x - y;
        i++;
    }
    return tolower((unsigned char)a[i]) -
           tolower((unsigned char)b[i]);
}

void sortIndices(char names[][LEN], int index[], int n) {
    int i, j, key;
    for (i = 0; i < n; i++) index[i] = i;

    for (i = 1; i < n; i++) {
        key = index[i];
        j = i - 1;
        while (j >= 0 &&
               compareIgnoreCase(names[index[j]], names[key]) > 0) {
            index[j + 1] = index[j];
            j--;
        }
        index[j + 1] = key;
    }
}

int binarySearch(char names[][LEN], int index[], int n,
                 const char key[]) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int result = compareIgnoreCase(names[index[mid]], key);
        if (result == 0) return index[mid];
        if (result < 0) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

```
int main(void) {
    char names[N][LEN] = {
        "Ravi", "Anu", "Kavin", "Anu", "Meena", "Bala"
    };
    int index[N], i, found;

    sortIndices(names, index, N);

    printf("Names in alphabetical order:\\n");
    for (i = 0; i < N; i++)
        printf("%s\\n", names[index[i]]);

    printf("Duplicate names:\\n");
    for (i = 1; i < N; i++) {
        if (compareIgnoreCase(names[index[i - 1]],
                              names[index[i]]) == 0)
            printf("%s\\n", names[index[i]]);
    }

    found = binarySearch(names, index, N, "Meena");
    if (found != -1)
        printf("Meena found in original array at index %d\\n", found);
    else
        printf("Meena not found\\n");

    return 0;
}
```

main.c

Clear Run

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define N 6
#define LEN 30

int compareIgnoreCase(const char a[], const char b[]) {
    int i = 0;
    while (a[i] && b[i]) {
        int x = tolower((unsigned char)a[i]);
        int y = tolower((unsigned char)b[i]);
        if (x != y) return x - y;
        i++;
    }
    return tolower((unsigned char)a[i]) -
           tolower((unsigned char)b[i]);
}

void sortIndices(char names[][LEN], int index[], int n) {
    int i, j, key;
    for (i = 0; i < n; i++) index[i] = i;

    for (i = 1; i < n; i++) {
        key = index[i];
        j = i - 1;
        while (j >= 0 &&
               compareIgnoreCase(names[index[j]], names[key]) > 0) {
            index[j + 1] = index[j];
            j--;
        }
        index[j + 1] = key;
    }
}

int binarySearch(char names[][LEN], int index[], int n,
                 const char key[]) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int result = compareIgnoreCase(names[index[mid]], key);
        if (result == 0) return index[mid];
        if (result < 0) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

```

Output

```
Names in alphabetical order:
Anu
Anu
Bala
Kavin
Meena
Ravi
Duplicate names:\nAnu
Meena found in original array at index 4
```

Q7 – Design an Election Counting Program from First Principles

1. Inputs, Outputs and Rules

Input: number of candidates c (at most 12) and v votes (at most 5,000 in the stated requirement). Each vote is a candidate number. A vote is valid if it is between 1 and c inclusive; otherwise it is invalid.

Output: valid vote count, invalid vote count, votes received by each candidate in descending order, winner and tie status.

2. Algorithm

```
ELECTION_COUNT(votes, v, c):
  count[1..c] = all zeros
  valid = 0; invalid = 0
  for each vote x:
    if 1 <= x <= c: count[x]++; valid++
    else: invalid++
  max_votes = maximum value in count
  winners = all candidates whose count equals max_votes
  if more than one winner: report tie
  else: report the single winner
  sort candidate numbers by count in descending order
  display counts and totals
```

3. Complexity

Counting uses one pass through the votes, so it takes O(v) time. Finding the maximum and identifying ties takes O(c). Sorting the candidate results takes O(c log c) with an efficient comparison sort. Total time is O(v + c log c), and the count array requires O(c) extra space.

4. Why a Count Array?

A count array indexed by candidate number updates a candidate's total directly in O(1) time. Searching a list of candidate names for every vote may take O(c) per vote, giving O(v·c) time. Sorting all votes and counting equal runs takes O(v log v) time. The count-array method is efficient when candidate identifiers are small integers in a known range.

5. Scaling and Limitation

For 500 candidates, increase the count array to 501 entries if candidate numbering begins at 1, and validate each vote against 1 through 500. One limitation is that the method assumes each vote contains a valid candidate number format; real systems also need input validation, secure vote handling and protection against duplicate or unauthorized voting.

```
#include <stdio.h>


#define MAX_CANDIDATES 12

#define MAX_VOTES 5000


int main(void) {

   int count[MAX_CANDIDATES + 1] = {0};

   int candidates[MAX_CANDIDATES];
```

```
int c, v, vote, valid = 0, invalid = 0;
int i, j, temp, maxVotes = 0, winnerCount = 0;

printf("Enter number of candidates (1-%d): ", MAX_CANDIDATES);
scanf("%d", &c);
if (c < 1 || c > MAX_CANDIDATES) {
   printf("Invalid number of candidates.\n");
   return 1;
}

printf("Enter number of votes (0-%d): ", MAX_VOTES);
scanf("%d", &v);
if (v < 0 || v > MAX_VOTES) {
   printf("Invalid number of votes.\n");
   return 1;
}

printf("Enter each vote as a candidate number (1 to %d):\n", c);
for (i = 0; i < v; i++) {
   scanf("%d", &vote);
   if (vote >= 1 && vote <= c) {
      count[vote]++;
      valid++;
   } else {
      invalid++;
   }
}

for (i = 1; i <= c; i++) {
   candidates[i - 1] = i;
   if (count[i] > maxVotes) maxVotes = count[i];
}

for (i = 1; i <= c; i++)
   if (count[i] == maxVotes) winnerCount++;
```

```
    for (i = 0; i < c - 1; i++) {
        for (j = i + 1; j < c; j++) {
            if (count[candidates[j]] > count[candidates[i]]) {
                temp = candidates[i];
                candidates[i] = candidates[j];
                candidates[j] = temp;
            }
        }
    }

    printf("\nValid votes: %d\n", valid);
    printf("Invalid votes: %d\n", invalid);
    printf("Candidate-wise results (descending):\n");
    for (i = 0; i < c; i++)
        printf("Candidate %d: %d votes\n",
               candidates[i], count[candidates[i]]);

    if (winnerCount > 1) {
        printf("Result: Tie between candidates");
        for (i = 1; i <= c; i++)
            if (count[i] == maxVotes) printf(" %d", i);
        printf("\n");
    } else {
        for (i = 1; i <= c; i++)
            if (count[i] == maxVotes)
                printf("Winner: Candidate %d\n", i);
    }
    return 0;
}
```

```c
#include <stdio.h>

#define MAX_CANDIDATES 12
#define MAX_VOTES 5000

int main(void) {
    int count[MAX_CANDIDATES + 1] = {0};
    int candidates[MAX_CANDIDATES];
    int c, v, vote, valid = 0, invalid = 0;
    int i, j, temp, maxVotes = 0, winnerCount = 0;

    printf("Enter number of candidates (1-%d): ", MAX_CANDIDATES);
    scanf("%d", &c);
    if (c < 1 || c > MAX_CANDIDATES) {
        printf("Invalid number of candidates.\n");
        return 1;
    }

    printf("Enter number of votes (0-%d): ", MAX_VOTES);
    scanf("%d", &v);
    if (v < 0 || v > MAX_VOTES) {
        printf("Invalid number of votes.\n");
        return 1;
    }

    printf("Enter each vote as a candidate number (1 to %d):\n", c);
    for (i = 0; i < v; i++) {
        scanf("%d", &vote);
        if (vote >= 1 && vote <= c) {
            count[vote]++;
            valid++;
        } else {
            invalid++;
        }
    }

    for (i = 1; i <= c; i++) {
        candidates[i - 1] = i;
        if (count[i] > maxVotes) maxVotes = count[i];
    }

    for (i = 1; i <= c; i++)
        if (count[i] == maxVotes) winnerCount++;
    for (i = 0; i < c - 1; i++) {
        for (j = i + 1; j < c; j++) {
            if (count[candidates[j]] > count[candidates[i]]) {
                temp = candidates[i];
                candidates[i] = candidates[j];
```

```
Enter number of candidates (1-12): 3
Enter number of votes (0-5000): 4
Enter each vote as a candidate number (1 to 3):
1 2 3 1

Valid votes: 4
Invalid votes: 0
Candidate-wise results (descending):
Candidate 1: 2 votes
Candidate 2: 1 votes
Candidate 3: 1 votes
Winner: Candidate 1
```